Roteiro de estudo
Márcio Fantini
Coltec-UFMG

MySQL básico

Consultas em uma única tabela

SELECT, filtros, agrupamentos

Material para aulas práticas e estudo dirigido
Formatado e Organizado com ajuda da Claude e ChatGPT

Sumário

1 Como utilizar este roteiro **1**
1.1 Etapas Comuns no Estudo do Mysql Básico 1

2 Preparação do banco de dados **2**
2.1 Criando e selecionando o banco 2
2.2 Criando as tabelas 2
2.3 Inserindo dados em clientes 3
2.4 Inserindo dados em vendas 3
2.5 Conferindo a estrutura 4

3 SELECT: escolhendo os dados **4**
3.1 Apelidos, ordenação e limitação 5

4 SELECT com WHERE **5**
4.1 Operadores úteis 6

5 WHERE com condições lógicas **7**
5.1 Precedência e parênteses 7

6 Funções de agregação sem GROUP BY **8**
6.1 COUNT: contagem 8
6.2 SUM, MAX e MIN 8

7 GROUP BY: formando grupos **9**
7.1 Agrupando por mais de uma coluna 10
7.2 WHERE antes do agrupamento 10
7.3 Um primeiro contato com HAVING 10

8 Funções de string **11**
8.1 UPPER e LOWER 11
8.2 CONCAT 11
8.3 LENGTH e CHAR_LENGTH 12
8.4 SUBSTRING, REPLACE e TRIM 12

9 Funções de data **12**
9.1 Data e hora atuais 13
9.2 Extraindo partes de uma data 13
9.3 Formatando datas 13

9.4 Diferença entre datas . . . 13
9.5 Datas e agrupamentos . . . 13

10 Ordem lógica de uma consulta 14

11 Revisão geral: estudo dirigido 15

12 Visão futura: funções de janela 15
12.1 ROW_NUMBER: numerando linhas . . . 15
12.2 RANK e DENSE_RANK: criando classificações . . . 16
12.3 Soma acumulada . . . 16
12.4 LAG: consultando a linha anterior . . . 17

13 Gabarito sugerido 17
13.1 Atividade 3.1 . . . 17
13.2 Atividade 4.1 . . . 17
13.3 Atividade 5.1 . . . 18
13.4 Atividade 6.1 . . . 18
13.5 Atividade 7.1 . . . 18
13.6 Atividade 8.1 . . . 19
13.7 Atividade 9.1 . . . 19
13.8 Revisão 11.1 — clientes . . . 20
13.9 Revisão 11.2 — vendas . . . 20

14 Checklist do estudante 21

1 Como utilizar este roteiro

Este material foi organizado para revermos os principais conceitos básicos (já vistos nas semanas 2 a 4). Vamos rever consultas básicas no MySQL. Execute os exemplos, observe os resultados e somente depois faça os exercícios propostos.

Objetivos de aprendizagem

Ao final do estudo, espera-se que o estudante consiga:

- selecionar todas ou apenas algumas colunas de uma tabela;
- filtrar linhas com `WHERE`;
- combinar condições com `AND`, `OR` e `NOT`;
- resumir dados com `COUNT`, `SUM`, `MAX` e `MIN`;
- formar grupos com `GROUP BY`;
- manipular textos e datas com funções do MySQL;

Atenção

Todos os exemplos deste roteiro consultam **uma única tabela por vez**. Não serão estudados, por enquanto, `JOIN`, subconsultas nem consultas correlacionadas. Esses assuntos exigem uma etapa posterior. Vide nossa apostila online.

1.1 Etapas Comuns no Estudo do Mysql Básico

Etapa	Conteúdo principal	Prática sugerida
1	Criação do banco, tabelas e dados; primeiros comandos `SELECT`	Seções 2 e 3
2	Filtros com `WHERE` e condições lógicas	Seções 4 e 5
3	Funções de agregação e `GROUP BY`	Seções 6 e 7
4	Funções de string	Seção 8
5	Funções de data	Seção 9

Nos exemplos das semanas anteriores nós carregamos os bancos (dados em arquivos .sql) diretamente no mysql. Nesse roteiro você vai criar o banco e as tabelas.

Repare que podemos criar um arquivo .sql e carregá-lo ao invés de digitar no mysql os comandos. Se quiser, experimente as duas formas...

2 Preparação do banco de dados

Usaremos uma pequena loja fictícia. A tabela `clientes` contém dados cadastrais. A tabela `vendas` registra cada venda com seu produto, categoria, valor e data.

2.1 Criando e selecionando o banco

Código 1: Criação do banco de dados

```sql
CREATE DATABASE loja_estudos
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE loja_estudos;
```

2.2 Criando as tabelas

Código 2: Estrutura da tabela clientes

```sql
CREATE TABLE clientes (
    id_cliente       INT PRIMARY KEY AUTO_INCREMENT,
    nome             VARCHAR(100) NOT NULL,
    cidade           VARCHAR(60) NOT NULL,
    estado           CHAR(2) NOT NULL,
    email            VARCHAR(120),
    data_nascimento  DATE,
    data_cadastro    DATE NOT NULL,
    ativo            BOOLEAN NOT NULL DEFAULT TRUE
);
```

Código 3: Estrutura da tabela vendas

```sql
CREATE TABLE vendas (
    id_venda       INT PRIMARY KEY AUTO_INCREMENT,
    id_cliente     INT NOT NULL,
    vendedor       VARCHAR(80) NOT NULL,
    produto        VARCHAR(100) NOT NULL,
    categoria      VARCHAR(40) NOT NULL,
    quantidade     INT NOT NULL,
    valor_unitario DECIMAL(10,2) NOT NULL,
    data_venda     DATE NOT NULL,
    forma_pagamento VARCHAR(30) NOT NULL
);
```

Por que existe o campo id_cliente em vendas?

Ele indica qual cliente realizou a compra. Em uma etapa futura, será possível combinar as duas tabelas. Neste roteiro, porém, cada consulta acessará somente `clientes` ou somente `vendas`; nenhuma combinação será realizada.

2.3 Inserindo dados em clientes

Código 4: Dados de exemplo da tabela clientes

```sql
INSERT INTO clientes
  (nome, cidade, estado, email, data_nascimento,
   data_cadastro, ativo)
VALUES
('Ana Souza',       'Belo Horizonte', 'MG', 'ana@exemplo.com',
 '1992-04-18', '2024-01-10', TRUE),
('Bruno Lima',      'Contagem',       'MG', 'bruno@exemplo.com',
 '1988-11-02', '2024-02-15', TRUE),
('Carla Mendes',    'São Paulo',      'SP', 'carla@exemplo.com',
 '1995-07-23', '2024-03-05', TRUE),
('Daniel Rocha',    'Rio de Janeiro', 'RJ', 'daniel@exemplo.com',
 '1981-01-30', '2024-03-27', FALSE),
('Elisa Martins',   'Belo Horizonte', 'MG', 'elisa@exemplo.com',
 '2000-09-12', '2024-04-08', TRUE),
('Fábio Costa',     'Salvador',       'BA', 'fabio@exemplo.com',
 '1979-06-05', '2024-05-19', TRUE),
('Gabriela Alves',  'Curitiba',       'PR', 'gabi@exemplo.com',
 '1997-12-15', '2024-06-11', FALSE),
('Henrique Nunes',  'São Paulo',      'SP', 'henrique@exemplo.com',
 '1990-03-08', '2024-07-22', TRUE),
('Isabela Freitas', 'Vitória',        'ES', 'isabela@exemplo.com',
 '1986-10-19', '2024-08-14', TRUE),
('João Ribeiro',    'Belo Horizonte', 'MG', 'joao@exemplo.com',
 '2001-02-27', '2024-09-03', TRUE),
('Karen Oliveira',  'Recife',         'PE', NULL,
 '1993-05-10', '2024-10-17', FALSE),
('Lucas Pereira',   'Contagem',       'MG', 'lucas@exemplo.com',
 '1984-08-21', '2024-11-09', TRUE);
```

2.4 Inserindo dados em vendas

Código 5: Dados de exemplo da tabela vendas

```sql
INSERT INTO vendas
  (id_cliente, vendedor, produto, categoria, quantidade,
   valor_unitario, data_venda, forma_pagamento)
VALUES
(1,  'Marina', 'Notebook Pro 14',   'Informática', 1, 4590.00, '2025-01-08', '
   Cartão'),
(2,  'Carlos', 'Mouse sem fio',     'Acessórios',  2,  120.00, '2025-01-10', 'Pix
   '),
(3,  'Marina', 'Monitor 24',        'Informática', 1,  980.00, '2025-01-15', '
   Cartão'),
(5,  'Rafael', 'Teclado mecânico',  'Acessórios',  1,  350.00, '2025-02-03', 'Pix
   '),
(1,  'Carlos', 'Mochila executiva', 'Acessórios',  1,  280.00, '2025-02-12', '
   Dinheiro'),
```

```
(6,  'Rafael', 'Smartphone X',      'Telefonia',   1, 2850.00, '2025-02-21', '
    Cartão'),
(8,  'Marina', 'Webcam HD',         'Acessórios',  2,  210.00, '2025-03-02', 'Pix
    '),
(9,  'Carlos', 'Tablet 10',         'Informática', 1, 1750.00, '2025-03-18', '
    Boleto'),
(10, 'Rafael', 'Fone Bluetooth',    'Telefonia',   3,  190.00, '2025-03-25', 'Pix
    '),
(12, 'Marina', 'Impressora Laser',  'Informática', 1, 1640.00, '2025-04-07', '
    Cartão'),
(2,  'Carlos', 'Cabo USB-C',        'Acessórios',  4,   45.00, '2025-04-16', '
    Dinheiro'),
(3,  'Rafael', 'Smartwatch Fit',    'Telefonia',   2,  590.00, '2025-04-29', '
    Cartão'),
(5,  'Marina', 'SSD 1 TB',          'Informática', 2,  520.00, '2025-05-06', 'Pix
    '),
(6,  'Carlos', 'Carregador portátil','Telefonia',  2,  160.00, '2025-05-20', '
    Boleto'),
(8,  'Rafael', 'Cadeira ergonômica','Escritório',  1, 1350.00, '2025-06-04', '
    Cartão'),
(9,  'Marina', 'Suporte notebook',  'Escritório',  2,  145.00, '2025-06-18', 'Pix
    '),
(10, 'Carlos', 'Mesa compacta',     'Escritório',  1,  890.00, '2025-07-01', '
    Boleto'),
(12, 'Rafael', 'Luminária LED',     'Escritório',  3,   95.00, '2025-07-12', 'Pix
    '),
(1,  'Marina', 'Headset Gamer',     'Acessórios',  1,  480.00, '2025-08-09', '
    Cartão'),
(5,  'Carlos', 'Notebook Essencial','Informática', 1, 3290.00, '2025-08-22', 'Pix
    ');
```

2.5 Conferindo a estrutura

Código 6: Comandos de inspeção

```
SHOW TABLES;
DESCRIBE clientes;
DESCRIBE vendas;
```

3 SELECT: escolhendo os dados

O comando `SELECT` recupera dados. O asterisco representa todas as colunas da tabela.

Código 7: Todas as colunas e todas as linhas

```
SELECT *
FROM clientes;
```

Em geral, é melhor informar apenas as colunas necessárias:

Código 8: Selecionando colunas específicas

```sql
SELECT nome, cidade, estado
FROM clientes;
```

3.1 Apelidos, ordenação e limitação

Código 9: Alias com AS

```sql
SELECT
    nome AS cliente,
    data_cadastro AS cadastro
FROM clientes;
```

Código 10: Ordenando e limitando o resultado

```sql
SELECT nome, cidade, estado
FROM clientes
ORDER BY nome ASC
LIMIT 5;
```

Conceito-chave

`ASC` ordena do menor para o maior ou de A a Z. `DESC` faz a ordenação inversa. `LIMIT` restringe a quantidade de linhas exibidas.

Atividade 3.1

Escreva consultas para:

a) exibir produto, categoria e valor unitário de todas as vendas;

b) listar os clientes em ordem alfabética inversa;

c) mostrar as três vendas mais recentes.

4 SELECT com WHERE

A cláusula `WHERE` mantém somente as linhas que satisfazem uma condição.

Código 11: Clientes de Minas Gerais

```sql
SELECT nome, cidade
FROM clientes
WHERE estado = 'MG';
```

Código 12: Comparação numérica

```sql
SELECT produto, quantidade, valor_unitario
FROM vendas
WHERE valor_unitario > 1000;
```

4.1 Operadores úteis

Operador	Finalidade	Exemplo
`=, <>, !=`	igualdade e diferença	`estado <> 'MG'`
`>, >=, <, <=`	comparações numéricas ou cronológicas	`quantidade >= 2`
`BETWEEN`	intervalo inclusivo	`valor BETWEEN 100 AND 500`
`IN`	pertence a uma lista	`estado IN ('MG','SP')`
`LIKE`	comparação por padrão	`nome LIKE 'A%'`
`IS NULL`	valor ausente	`email IS NULL`

Código 13: Intervalo com BETWEEN

```sql
SELECT produto, valor_unitario
FROM vendas
WHERE valor_unitario BETWEEN 200 AND 1000;
```

Código 14: Lista de valores com IN

```sql
SELECT nome, cidade, estado
FROM clientes
WHERE estado IN ('MG', 'SP', 'RJ');
```

Código 15: Padrões com LIKE

```sql
-- Nomes que começam com a letra A
SELECT nome
FROM clientes
WHERE nome LIKE 'A%';

-- Produtos que contêm a palavra Notebook
SELECT produto
FROM vendas
WHERE produto LIKE '%Notebook%';
```

Código 16: Teste correto de valores nulos

```sql
SELECT nome, email
FROM clientes
WHERE email IS NULL;
```

Atenção

Não use `email = NULL`. Como `NULL` representa ausência ou valor desconhecido, o teste correto é `IS NULL`; o inverso é `IS NOT NULL`.

Atividade 4.1

Consulte:

a) clientes cadastrados depois de `2024-06-30`;

b) vendas cuja forma de pagamento seja `Pix`;

c) produtos com valor unitário entre 100 e 500;

d) clientes cujo nome termina com a letra `a`;

e) clientes que possuem e-mail cadastrado.

5 WHERE com condições lógicas

Os operadores lógicos permitem montar filtros compostos.

Operador	Quando a condição é verdadeira
`AND`	todas as condições ligadas por ele são verdadeiras;
`OR`	pelo menos uma das condições é verdadeira;
`NOT`	a condição seguinte não é verdadeira.

Código 17: Condição com AND

```sql
SELECT nome, cidade, estado
FROM clientes
WHERE estado = 'MG'
  AND ativo = TRUE;
```

Código 18: Condição com OR

```sql
SELECT produto, categoria, forma_pagamento
FROM vendas
WHERE forma_pagamento = 'Pix'
   OR forma_pagamento = 'Dinheiro';
```

Código 19: Condição com NOT

```sql
SELECT nome, estado
FROM clientes
WHERE NOT estado = 'MG';
```

5.1 Precedência e parênteses

O MySQL avalia `AND` antes de `OR`. Parênteses tornam a intenção explícita e evitam resultados inesperados.

Código 20: Agrupando condições com parênteses

```sql
SELECT produto, categoria, valor_unitario, forma_pagamento
FROM vendas
WHERE categoria = 'Informática'
  AND (forma_pagamento = 'Pix'
       OR forma_pagamento = 'Cartão');
```

Leia o resultado

Primeiro é verificada a expressão entre parênteses. A venda deve ter sido paga por Pix ou Cartão e, além disso, pertencer à categoria Informática.

Atividade 5.1

Liste:

a) clientes ativos de Minas Gerais ou São Paulo;

b) vendas da categoria Acessórios com quantidade maior ou igual a 2;

c) vendas de Informática ou Telefonia realizadas por Marina;

d) clientes que não estejam inativos e tenham e-mail cadastrado.

6 Funções de agregação sem GROUP BY

Funções de agregação resumem várias linhas em um único resultado.

6.1 COUNT: contagem

Código 21: Contando linhas

```sql
SELECT COUNT(*) AS total_clientes
FROM clientes;
```

Código 22: COUNT com uma coluna

```sql
SELECT COUNT(email) AS clientes_com_email
FROM clientes;
```

COUNT(*) e COUNT(coluna)

`COUNT(*)` conta todas as linhas. `COUNT(email)` conta apenas as linhas em que `email` não é `NULL`.

6.2 SUM, MAX e MIN

O total de cada item vendido é dado por `quantidade * valor_unitario`.

Código 23: Somatório de todas as vendas

```sql
SELECT
    SUM(quantidade * valor_unitario) AS faturamento_total
FROM vendas;
```

Código 24: Maior e menor preço unitário

```sql
SELECT
    MAX(valor_unitario) AS maior_preco,
    MIN(valor_unitario) AS menor_preco
FROM vendas;
```

É possível filtrar as linhas **antes** de agregá-las:

Código 25: Agregação com filtro

```sql
SELECT
    COUNT(*) AS vendas_no_pix,
    SUM(quantidade * valor_unitario) AS total_no_pix
FROM vendas
WHERE forma_pagamento = 'Pix';
```

Atividade 6.1

Descubra:

a) quantos clientes estão ativos;

b) quantas unidades de produtos foram vendidas no total;

c) o maior e o menor valor unitário entre as vendas de Acessórios;

d) o faturamento obtido por Marina;

e) quantas cidades diferentes aparecem na tabela clientes.

Dica para o item e: pesquise o efeito de `DISTINCT` dentro de `COUNT`.

7 GROUP BY: formando grupos

Sem `GROUP BY`, uma função de agregação resume todo o conjunto filtrado. Com `GROUP BY`, o MySQL separa as linhas em grupos e calcula um resultado para cada grupo.

Código 26: Quantidade de clientes por estado

```sql
SELECT
    estado,
    COUNT(*) AS total_clientes
FROM clientes
GROUP BY estado
ORDER BY total_clientes DESC;
```

Código 27: Resumo de vendas por categoria

```sql
SELECT
    categoria,
    COUNT(*) AS numero_de_vendas,
    SUM(quantidade) AS unidades,
    SUM(quantidade * valor_unitario) AS faturamento,
    MAX(valor_unitario) AS maior_preco,
    MIN(valor_unitario) AS menor_preco
FROM vendas
GROUP BY categoria
ORDER BY faturamento DESC;
```

7.1 Agrupando por mais de uma coluna

Código 28: Vendas por vendedor e forma de pagamento

```sql
SELECT
    vendedor,
    forma_pagamento,
    COUNT(*) AS quantidade_de_vendas
FROM vendas
GROUP BY vendedor, forma_pagamento
ORDER BY vendedor, quantidade_de_vendas DESC;
```

7.2 WHERE antes do agrupamento

Código 29: Agrupando somente as vendas do primeiro semestre

```sql
SELECT
    vendedor,
    SUM(quantidade * valor_unitario) AS faturamento
FROM vendas
WHERE data_venda BETWEEN '2025-01-01' AND '2025-06-30'
GROUP BY vendedor
ORDER BY faturamento DESC;
```

Atenção

Em uma consulta agrupada, cada coluna exibida no `SELECT` deve normalmente estar no `GROUP BY` ou dentro de uma função de agregação. Essa regra evita resultados ambíguos e é exigida pelo modo `ONLY_FULL_GROUP_BY`.

7.3 Um primeiro contato com HAVING

`WHERE` filtra linhas antes do agrupamento. `HAVING` filtra os grupos depois que as funções de agregação foram calculadas.

Código 30: Categorias com faturamento superior a 2000

```sql
SELECT
    categoria,
    SUM(quantidade * valor_unitario) AS faturamento
FROM vendas
GROUP BY categoria
HAVING SUM(quantidade * valor_unitario) > 2000
ORDER BY faturamento DESC;
```

Atividade 7.1

Produza as seguintes sínteses:

a) total de clientes por cidade;

b) total vendido por forma de pagamento;

c) maior e menor preço unitário por categoria;

d) quantidade de vendas e faturamento por vendedor;

e) vendedores com faturamento superior a 5.000;

f) quantidade de clientes ativos e inativos, agrupados pelo campo `ativo`.

8 Funções de string

Funções de string transformam ou analisam textos. Elas podem ser usadas no `SELECT` sem alterar o conteúdo armazenado na tabela.

8.1 UPPER e LOWER

Código 31: Alterando maiúsculas e minúsculas na exibição

```sql
SELECT
    nome,
    UPPER(nome) AS nome_maiusculo,
    LOWER(cidade) AS cidade_minuscula
FROM clientes;
```

8.2 CONCAT

Código 32: Concatenando textos

```sql
SELECT
    nome,
    CONCAT(cidade, ' - ', estado) AS localidade
FROM clientes;
```

8.3 LENGTH e CHAR_LENGTH

Código 33: Comprimento em bytes e em caracteres

```sql
SELECT
    nome,
    LENGTH(nome) AS total_bytes,
    CHAR_LENGTH(nome) AS total_caracteres
FROM clientes;
```

Conceito-chave

`LENGTH` conta bytes; `CHAR_LENGTH` conta caracteres. Em textos UTF-8, uma letra acentuada pode ocupar mais de um byte. Para contar letras, prefira `CHAR_LENGTH`.

8.4 SUBSTRING, REPLACE e TRIM

Código 34: Outras transformações de texto

```sql
SELECT
    produto,
    SUBSTRING(produto, 1, 10) AS primeiros_10_caracteres,
    REPLACE(produto, 'Notebook', 'Laptop') AS nome_alternativo
FROM vendas;
```

Código 35: Removendo espaços nas extremidades

```sql
SELECT
    TRIM('   MySQL básico   ') AS texto_limpo;
```

Atividade 8.1

Crie consultas que:

a) exibam o nome do cliente em maiúsculas;

b) montem a frase "Ana Souza mora em Belo Horizonte/MG", adaptada a cada linha;

c) mostrem produto e quantidade de caracteres do nome do produto;

d) substituam, apenas no resultado, a palavra `Cartão` por `Crédito`;

e) exibam as três primeiras letras do nome de cada vendedor.

9 Funções de data

No MySQL, datas no formato padrão são escritas como `AAAA-MM-DD`.

9.1 Data e hora atuais

Código 36: Consultando o relógio do servidor

```
SELECT
    CURDATE() AS data_atual,
    NOW() AS data_e_hora_atual;
```

9.2 Extraindo partes de uma data

Código 37: Ano, mês e dia

```
SELECT
    produto,
    data_venda,
    YEAR(data_venda) AS ano,
    MONTH(data_venda) AS mes,
    DAY(data_venda) AS dia
FROM vendas;
```

9.3 Formatando datas

Código 38: Data no formato brasileiro

```
SELECT
    nome,
    DATE_FORMAT(data_cadastro, '%d/%m/%Y') AS cadastro_br
FROM clientes;
```

9.4 Diferença entre datas

Código 39: Dias decorridos desde o cadastro

```
SELECT
    nome,
    data_cadastro,
    DATEDIFF(CURDATE(), data_cadastro) AS dias_desde_cadastro
FROM clientes
ORDER BY dias_desde_cadastro DESC;
```

9.5 Datas e agrupamentos

Código 40: Faturamento por mês

```
SELECT
    YEAR(data_venda) AS ano,
    MONTH(data_venda) AS mes,
```

```
    SUM(quantidade * valor_unitario) AS faturamento
FROM vendas
GROUP BY YEAR(data_venda), MONTH(data_venda)
ORDER BY ano, mes;
```

Código 41: Rótulo mensal amigável

```
SELECT
    DATE_FORMAT(data_venda, '%Y-%m') AS mes,
    COUNT(*) AS total_vendas,
    SUM(quantidade * valor_unitario) AS faturamento
FROM vendas
GROUP BY DATE_FORMAT(data_venda, '%Y-%m')
ORDER BY mes;
```

Atividade 9.1

Escreva consultas para:

a) exibir nome, nascimento e ano de nascimento dos clientes;

b) listar somente vendas realizadas em abril de 2025;

c) mostrar as datas de venda no formato brasileiro;

d) contar quantas vendas ocorreram em cada mês;

e) calcular quantos dias se passaram entre cada venda e `2025-12-31`.

10 Ordem lógica de uma consulta

Embora os comandos sejam escritos começando por `SELECT`, uma forma útil de entender o processamento conceitual é:

Ordem	Cláusula	Papel
1	`FROM`	escolhe a tabela;
2	`WHERE`	filtra as linhas;
3	`GROUP BY`	forma os grupos;
4	`HAVING`	filtra os grupos;
5	`SELECT`	calcula e escolhe as colunas;
6	`ORDER BY`	ordena o resultado;
7	`LIMIT`	limita a quantidade exibida.

Modelo mental

Pense assim: pegue uma tabela, descarte linhas, forme grupos, descarte grupos, escolha o que será mostrado, ordene e limite o resultado.

11 Revisão geral: estudo dirigido

Resolva sem consultar o gabarito. Todos os problemas podem ser solucionados com uma única tabela.

Atividade 11.1 — clientes

1. Liste nome, cidade e estado dos clientes ativos.
2. Liste os clientes de MG e SP em ordem alfabética.
3. Mostre os clientes cadastrados no segundo semestre de 2024.
4. Conte quantos clientes existem em cada estado.
5. Mostre apenas os estados que possuem pelo menos dois clientes.
6. Exiba o nome em maiúsculas e a localidade no formato `Cidade/UF`.
7. Exiba o ano de nascimento e conte clientes por ano de nascimento.

Atividade 11.2 — vendas

1. Liste as vendas pagas por Pix com valor unitário superior a 200.
2. Calcule o valor total de cada item vendido, sem agrupar.
3. Calcule o faturamento total da loja.
4. Calcule o faturamento de cada vendedor.
5. Identifique o maior e o menor preço unitário por categoria.
6. Conte as vendas de cada forma de pagamento.
7. Mostre o faturamento mensal em ordem cronológica.
8. Liste somente categorias cujo faturamento supere 2.000.

12 Visão futura: funções de janela

Funções de janela são um assunto mais avançado. Elas calculam valores usando um conjunto de linhas relacionadas, mas **preservam cada linha do resultado**. Isso as diferencia de `GROUP BY`, que reduz várias linhas a uma linha por grupo.

Atenção

Esta seção é apenas uma apresentação. Os exemplos pressupõem MySQL 8.0 ou superior e serão estudados com mais profundidade futuramente.

12.1 ROW_NUMBER: numerando linhas

Código 42: Numeração das vendas da mais cara para a mais barata

```sql
SELECT
    produto,
    valor_unitario,
    ROW_NUMBER() OVER (
        ORDER BY valor_unitario DESC
    ) AS numero_da_linha
FROM vendas;
```

12.2 RANK e DENSE_RANK: criando classificações

Código 43: Ranking de preços dentro de cada categoria

```sql
SELECT
    categoria,
    produto,
    valor_unitario,
    RANK() OVER (
        PARTITION BY categoria
        ORDER BY valor_unitario DESC
    ) AS posicao,
    DENSE_RANK() OVER (
        PARTITION BY categoria
        ORDER BY valor_unitario DESC
    ) AS posicao_sem_saltos
FROM vendas;
```

RANK versus DENSE_RANK

Quando há empate, `RANK` pode deixar uma lacuna na próxima posição. `DENSE_RANK` não deixa essa lacuna. Já `ROW_NUMBER` sempre atribui números diferentes, mesmo quando os valores empatam.

12.3 Soma acumulada

Código 44: Faturamento acumulado ao longo do tempo

```sql
SELECT
    data_venda,
    produto,
    quantidade * valor_unitario AS total_item,
    SUM(quantidade * valor_unitario) OVER (
        ORDER BY data_venda, id_venda
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS faturamento_acumulado
FROM vendas
ORDER BY data_venda, id_venda;
```

12.4 LAG: consultando a linha anterior

Código 45: Comparação com o preço da venda anterior

```sql
SELECT
    data_venda,
    produto,
    valor_unitario,
    LAG(valor_unitario) OVER (
        ORDER BY data_venda, id_venda
    ) AS preco_anterior
FROM vendas
ORDER BY data_venda, id_venda;
```

Leia o resultado

Mais adiante, as funções de janela permitirão responder perguntas como: "qual é a posição de cada venda?", "quanto já foi faturado até esta data?" e "qual era o valor na linha anterior?" sem eliminar os registros individuais.

13 Gabarito sugerido

Há várias soluções corretas. As consultas abaixo apresentam possibilidades compatíveis com o conteúdo estudado.

13.1 Atividade 3.1

```sql
SELECT produto, categoria, valor_unitario FROM vendas;

SELECT nome FROM clientes ORDER BY nome DESC;

SELECT id_venda, produto, data_venda
FROM vendas
ORDER BY data_venda DESC, id_venda DESC
LIMIT 3;
```

13.2 Atividade 4.1

```sql
SELECT nome, data_cadastro FROM clientes
WHERE data_cadastro > '2024-06-30';

SELECT * FROM vendas WHERE forma_pagamento = 'Pix';

SELECT produto, valor_unitario FROM vendas
WHERE valor_unitario BETWEEN 100 AND 500;

SELECT nome FROM clientes WHERE nome LIKE '%a';
```

```sql

SELECT nome, email FROM clientes WHERE email IS NOT NULL;
```

13.3 Atividade 5.1

```sql
SELECT nome, estado FROM clientes
WHERE ativo = TRUE AND estado IN ('MG', 'SP');

SELECT produto, quantidade FROM vendas
WHERE categoria = 'Acessórios' AND quantidade >= 2;

SELECT vendedor, produto, categoria FROM vendas
WHERE vendedor = 'Marina'
  AND categoria IN ('Informática', 'Telefonia');

SELECT nome, email FROM clientes
WHERE NOT ativo = FALSE AND email IS NOT NULL;
```

13.4 Atividade 6.1

```sql
SELECT COUNT(*) AS ativos FROM clientes WHERE ativo = TRUE;

SELECT SUM(quantidade) AS unidades FROM vendas;

SELECT MAX(valor_unitario), MIN(valor_unitario)
FROM vendas WHERE categoria = 'Acessórios';

SELECT SUM(quantidade * valor_unitario) AS faturamento_marina
FROM vendas WHERE vendedor = 'Marina';

SELECT COUNT(DISTINCT cidade) AS cidades FROM clientes;
```

13.5 Atividade 7.1

```sql
SELECT cidade, COUNT(*) AS total
FROM clientes GROUP BY cidade ORDER BY total DESC;

SELECT forma_pagamento,
       SUM(quantidade * valor_unitario) AS total
FROM vendas GROUP BY forma_pagamento ORDER BY total DESC;

SELECT categoria, MAX(valor_unitario), MIN(valor_unitario)
FROM vendas GROUP BY categoria;

SELECT vendedor, COUNT(*) AS vendas,
       SUM(quantidade * valor_unitario) AS faturamento
FROM vendas GROUP BY vendedor ORDER BY faturamento DESC;
```

```sql

SELECT vendedor,
       SUM(quantidade * valor_unitario) AS faturamento
FROM vendas
GROUP BY vendedor
HAVING SUM(quantidade * valor_unitario) > 5000;

SELECT ativo, COUNT(*) AS total
FROM clientes GROUP BY ativo;
```

13.6 Atividade 8.1

```sql
SELECT UPPER(nome) AS nome FROM clientes;

SELECT CONCAT(nome, ' mora em ', cidade, '/', estado) AS frase
FROM clientes;

SELECT produto, CHAR_LENGTH(produto) AS caracteres FROM vendas;

SELECT forma_pagamento,
       REPLACE(forma_pagamento, 'Cartão', 'Crédito') AS exibicao
FROM vendas;

SELECT vendedor, SUBSTRING(vendedor, 1, 3) AS iniciais
FROM vendas;
```

13.7 Atividade 9.1

```sql
SELECT nome, data_nascimento, YEAR(data_nascimento) AS ano
FROM clientes;

SELECT * FROM vendas
WHERE data_venda BETWEEN '2025-04-01' AND '2025-04-30';

SELECT produto, DATE_FORMAT(data_venda, '%d/%m/%Y') AS data_br
FROM vendas;

SELECT YEAR(data_venda) AS ano, MONTH(data_venda) AS mes,
       COUNT(*) AS total
FROM vendas
GROUP BY YEAR(data_venda), MONTH(data_venda)
ORDER BY ano, mes;

SELECT produto, data_venda,
       DATEDIFF('2025-12-31', data_venda) AS dias
FROM vendas;
```

13.8 Revisão 11.1 — clientes

```sql
SELECT nome, cidade, estado FROM clientes WHERE ativo = TRUE;

SELECT nome, cidade, estado FROM clientes
WHERE estado IN ('MG', 'SP') ORDER BY nome;

SELECT nome, data_cadastro FROM clientes
WHERE data_cadastro BETWEEN '2024-07-01' AND '2024-12-31';

SELECT estado, COUNT(*) AS total
FROM clientes GROUP BY estado ORDER BY total DESC;

SELECT estado, COUNT(*) AS total
FROM clientes GROUP BY estado HAVING COUNT(*) >= 2;

SELECT UPPER(nome) AS nome,
       CONCAT(cidade, '/', estado) AS localidade
FROM clientes;

SELECT YEAR(data_nascimento) AS ano, COUNT(*) AS total
FROM clientes
GROUP BY YEAR(data_nascimento)
ORDER BY ano;
```

13.9 Revisão 11.2 — vendas

```sql
SELECT * FROM vendas
WHERE forma_pagamento = 'Pix' AND valor_unitario > 200;

SELECT produto, quantidade, valor_unitario,
       quantidade * valor_unitario AS total_item
FROM vendas;

SELECT SUM(quantidade * valor_unitario) AS faturamento
FROM vendas;

SELECT vendedor,
       SUM(quantidade * valor_unitario) AS faturamento
FROM vendas GROUP BY vendedor ORDER BY faturamento DESC;

SELECT categoria, MAX(valor_unitario), MIN(valor_unitario)
FROM vendas GROUP BY categoria;

SELECT forma_pagamento, COUNT(*) AS total
FROM vendas GROUP BY forma_pagamento ORDER BY total DESC;

SELECT DATE_FORMAT(data_venda, '%Y-%m') AS mes,
       SUM(quantidade * valor_unitario) AS faturamento
FROM vendas
GROUP BY DATE_FORMAT(data_venda, '%Y-%m')
```

```sql
ORDER BY mes;

SELECT categoria,
       SUM(quantidade * valor_unitario) AS faturamento
FROM vendas
GROUP BY categoria
HAVING SUM(quantidade * valor_unitario) > 2000;
```

14 Checklist do estudante

Marque cada item quando conseguir explicá-lo e criar um exemplo próprio.

- ○ Sei escolher colunas com `SELECT`.
- ○ Sei ordenar com `ORDER BY` e limitar com `LIMIT`.
- ○ Sei filtrar com `WHERE`.
- ○ Sei usar `BETWEEN`, `IN`, `LIKE` e `IS NULL`.
- ○ Sei combinar condições com `AND`, `OR` e `NOT`.
- ○ Entendo a diferença entre `COUNT(*)` e `COUNT(coluna)`.
- ○ Sei usar `COUNT`, `SUM`, `MAX` e `MIN`.
- ○ Sei criar resumos com `GROUP BY`.
- ○ Entendo a diferença entre `WHERE` e `HAVING`.
- ○ Sei aplicar funções de string sem alterar os dados armazenados.
- ○ Sei extrair, formatar e comparar datas.
- ○ Consigo explicar, de maneira inicial, o que uma função de janela faz.

Próxima etapa sugerida

Relacionamentos, chaves estrangeiras e `JOIN`. Depois: subconsultas, índices, views e estudo aprofundado das funções de janela.